

OEM Compliance Mapping System

Architecture and tech stack overview

This is a quick writeup of how the project in the zip file is put together. It's an RFP-to-product compliance tool: you give it a folder of vendor datasheets and an RFP document, and it pulls out the requirements, checks them against the datasheets, and produces a scored report showing which product matches best, with evidence for each requirement.

Everything runs locally on one machine. There's no cloud database or hosted LLM involved — it talks to a self-hosted Ollama server on the local network for embeddings and chat completions, and stores everything in ChromaDB on disk.

1. What the system does

There are really two knowledge bases that work together. One is a library of OEM datasheets (firewalls, switches, etc. from vendors like Fortinet, Palo Alto, Cisco, Juniper) that gets built up once and reused across multiple RFPs. The other is built fresh each time: the requirements pulled out of a specific RFP. The compliance engine compares the two and scores each candidate product.

It produces three things:

- A searchable vector knowledge base of every datasheet that's been ingested.
- A structured, de-duplicated list of requirements from the RFP, tagged mandatory or optional.
- A ranked compliance report (Markdown and JSON) per product, with page-level evidence for every requirement.

2. How data moves through the system

There are two separate pipelines feeding into one scoring engine. The first runs once per datasheet. The second runs once per RFP. Both use the same embedding model, so the vectors they produce live in the same space and can be compared directly.

A. Datasheet ingestion (builds the product knowledge base)

Step	Module	What it does
Extract	pdf_extractor.py	Pulls text and tables per page using pdfplumber and PyMuPDF. If a page has no real text layer (a scanned spec sheet), it falls back to OCR via Tesseract or PaddleOCR.

Identify models	model_identifier.py	Finds every product model number on the page using regex patterns and section heuristics, and splits multi-model comparison tables into per-model spec dictionaries.
Chunk	chunker.py	Converts each model's specs into purpose-built chunks (spec table, features, ordering info, description), sized 800-1200 characters.
Embed and store	vector_store.py	Each chunk is embedded with bge-m3 (via Ollama) and stored in a ChromaDB collection, tagged with vendor, model, chunk type, and source page.

B. RFP requirement extraction (runs once per submission)

Step	Module	What it does
Read	rfp_extractor.py	User picks a page range in the UI. Those pages are pulled and split into roughly 2,500-character chunks.
Extract	rfp_extractor.py	Two passes — a regex pass catches plain numeric thresholds for free, then an LLM pass (using a fast, non-reasoning model) catches everything else as structured JSON.
Dedup and store	rfp_extractor.py	Requirements are assigned IDs, tagged mandatory or optional, written to a JSON file, and embedded into their own per-RFP Chroma collection.

C. Compliance scoring (the matching engine)

Step	Module	What it does
Retrieve	retrieval.py	For each requirement, a semantic search returns the most relevant OEM chunks, grouped by vendor and model.
Judge	matcher.py	Each requirement/product pair is evaluated and given a verdict (Full Match, Partial Match, No Match). Several cheap checks — no evidence at all, a plain numeric comparison, a very high similarity score — short-circuit before any LLM call is made.
Rank	ranker.py	Per-product scores are weighted so mandatory requirements count twice as much as optional ones. Products are evaluated in parallel using a thread pool.

Report	reporter.py	Writes a ranked Markdown report (summary, compliance matrix, gap analysis, cited evidence) plus the raw JSON.
--------	-------------	---

The matcher is tuned to avoid LLM calls where it can — it's built for a CPU-only machine, not a GPU box. Roughly a fifth to a quarter of requirements (the ones that are just numeric thresholds) never touch the LLM at all.

3. Module map

The code is organized by responsibility, with a thin API and UI layer on top of four backend packages:

File / folder	Role
app.py	Streamlit UI with three tabs: Upload RFP, Manage Database, Reports.
main.py	Command-line interface — ingest, search, stats, delete, list-docs.
api.py	OEMKnowledgeBase — the clean entry point other modules import.
ingestion/	PDF extraction, model identification, chunking, OCR fallback.
rfp/	RFP page reading and requirement extraction (regex + LLM).
compliance/	Retrieval, matching, ranking, and report generation — the scoring engine.
knowledge_base/	VectorStoreManager, the ChromaDB wrapper used by both knowledge bases.
services/	Thin clients — embedding_service.py for the Ollama embed API, llm_services.py for the OpenAI-compatible chat client.
models/ compliance/schemas.py	Pydantic data models shared across every stage.
config/settings.py	All the tunables in one place — chunk sizes, model names, endpoints, thresholds.
ui/kb_inventory.py	Read-only helper that builds the “Manage Database” view from chunk metadata.

One thing worth mentioning: section_rules.py exists purely to fix a real bug. The logic for deciding whether a section is “family-level” (shared across all models) or per-model used to be copy-pasted in both chunker.py and model_identifier.py, and the two copies had drifted apart. It's now a single shared module both files import from.

4. Tech stack

Core

- **Language:** Python (the repo has compiled bytecode for both 3.10 and 3.14, so it looks like development happens on 3.14 locally and gets deployed on 3.10).
- **UI:** Streamlit — a three-tab app for uploading datasheets/RFPs, managing the knowledge base, and browsing reports. No separate frontend build.
- **CLI:** argparse, with the same functionality as the UI for scripted or headless ingestion.
- **Data models:** Pydantic v2 for every object passed between stages (chunks, requirements, evidence, reports).
- **Config:** Python dataclasses plus python-dotenv. One config tree, with .env able to override things like the LLM API key.
- **Logging:** loguru in the ingestion path, plain logging in the compliance engine.

Document processing

- **PDF text:** pdfplumber and PyMuPDF (fitz), used together — pdfplumber for layout-aware text and tables, PyMuPDF as a faster fallback / cross-check.
- **OCR:** Tesseract, with PaddleOCR as an optional alternative. Only triggered when a page has no real text layer (checked via pdffonts or a PyMuPDF font probe).
- **Image handling:** Pillow, plus OpenCV when available for OCR preprocessing. The pipeline degrades gracefully without OpenCV.

AI / retrieval

- **Embeddings:** bge-m3, served through Ollama. The same model embeds both datasheet chunks and RFP requirements, which is what makes cosine similarity between the two meaningful.
- **Vector store:** ChromaDB, persistent and local. One collection for the OEM knowledge base, one sub-collection per RFP. No external vector database service.
- **LLM runtime:** Ollama, self-hosted on the local network. The app talks to it through the openai Python SDK pointed at a local base_url.
- **Model routing:** gemma2:9b by default, configurable. A reasoning model is used for compliance verdicts and report text; a separate “fast” model slot is reserved for pure-JSON extraction work.

On that last point — the LLM client exposes two call paths. `generate()` goes to the reasoning model and is used for compliance verdicts, where the extra deliberation is worth the time. `generate_fast()` goes to a separate non-reasoning model (or suppresses chain-of-thought on the same model) for RFP requirement extraction, where all that matters is clean JSON output, quickly. On a CPU-only machine this is the difference between a call taking minutes versus

seconds.

```
class LLMService:
    def generate(self, prompt):          # reasoning model
        ... # compliance verdicts, report text

    def generate_fast(self, prompt):     # non-reasoning model
        ... # RFP requirement extraction (pure JSON)
```

5. Data model and storage

Everything lives on local disk under a single data/ directory — there's no external database server to set up.

```
data/
  raw/                source PDFs (datasheets), mirrored on ingest
  processed/          intermediate extraction artifacts
  vector_store/       ChromaDB persistent store
    oem_datasheets    (shared product knowledge base)
    rfp_requirements_* (one collection per RFP)
  requirements/       extracted RFP requirements, as JSON
  compliance_reports/ generated .md + .json reports
  logs/              loguru output
```

Key schema objects

Object	Carries
DocumentChunk	text, vendor, model_name, chunk_type, source page, embedding
ModelSpec	per-product spec dict, derived from one or more comparison tables
Requirement	requirement text, mandatory flag, source page, generated ID
EvidenceChunk	the KB chunk that backs a verdict, with similarity score
RequirementResult	verdict (Full Match / Partial / No Match / Not Found / etc.) plus evidence
ProductScore	per-product rollup — mandatory score, overall score, rank

Documents are keyed by a 16-character SHA-256 prefix of the file content, so re-ingesting the same PDF is a no-op unless --force is passed.

6. A few design choices worth knowing about

Everything runs local. No managed vector database, no cloud LLM API by default.

ChromaDB persists to disk and Ollama handles both embeddings and chat completions from

a box on the LAN. This keeps vendor datasheets and RFP content off third-party infrastructure.

Two models instead of one. A reasoning model and a fast extraction model are configured separately. The comments in `llm_services.py` and `rfp_extractor.py` both describe cutting multi-minute calls down to seconds by avoiding chain-of-thought on the extraction path — this was clearly tuned against real latency problems.

Cheap checks before expensive ones. The matcher tries to resolve a verdict with zero LLM calls first: no evidence at all gets an instant “Not Found,” a plain numeric threshold gets resolved by regex, and a very high similarity match with no unit to check gets an instant “Full Match.” The LLM is the last resort, not the first.

Shared embedding space. Datasheet chunks and RFP requirements are embedded with the same bge-m3 model specifically so that cosine similarity between the two means something — this is what makes semantic retrieval across the two collections work at all.

One source of truth for fragile logic. `section_rules.py` was pulled out after a bug where two files had silently drifted copies of the same classification logic. It's a small thing, but it's a sign the codebase has been through at least one real debugging cycle, not just written once and shipped.

(Pulled this together from the actual source — docstrings, inline comments, and config defaults — rather than any external docs.)